IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received 27 May 2025, accepted 22 June 2025, date of publication 30 June 2025, date of current version 8 July 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3584373

RESEARCH ARTICLE

Optimized Ensemble Deep Learning for Real-Time Intrusion Detection on Resource-Constrained Raspberry Pi Devices

MUHAMMAD BISRI MUSTHAFA[1], SAMSUL HUDA[2], (Member, IEEE), TUY TAN NGUYEN[3], (Senior Member, IEEE), YUTA KODERA[1], (Member, IEEE), AND YASUYUKI NOGAMI[1], (Member, IEEE)

[1]Graduate School of Environmental, Life, Natural Science and Technology, Okayama University, Okayama 700-8530, Japan
[2]Interdisciplinary Education and Research Field, Okayama University, Okayama 700-8530, Japan
[3]School of Informatics, Computing, and Cyber Systems, Northern Arizona University, Flagstaff, AZ 86011, USA

Corresponding authors: Samsul Huda (shuda@okayama-u.ac.jp) and Yasuyuki Nogami (yasuyuki.nogami@okayama-u.ac.jp)

ABSTRACT The rapid growth of Internet of Things (IoT) networks has increased security risks, making it essential to have effective Intrusion Detection Systems (IDS) for real-time threat detection. Deep learning techniques offer promising solutions for such detection due to their superior complex pattern recognition and anomaly detection capabilities in large datasets. This paper proposes an optimized ensemble-based IDS designed specifically for efficient deployment on edge hardware. However, deploying such computationally intensive models on resource-limited edge devices remains a significant challenge due to model size and computational overhead on devices with limited processing capabilities. Building upon our previously developed stacked Long Short-Term Memory (LSTM) model integrated with ANOVA feature selection, we optimize it by integrating dual-stage model compression: pruning and quantization to create a lightweight model suitable for real-time inference on Raspberry Pi devices. To evaluate the system under realistic conditions, we combined with a Kafka-based testbed to simulate dynamic IoT environments with variable traffic loads, delays, and multiple simultaneous attack sources. This enables the assessment of detection performance under varying traffic volumes, latency, and overlapping attack scenarios. The proposed system maintains high detection performance with accuracy of 97.3% across all test scenarios, while efficiently leveraging multi-core processing with peak CPU usage reaching 111.8%. These results demonstrate the system's practical viability for real-time IoT security at the edge.

INDEX TERMS Internet of things, intrusion detection system, stacked lstm, pruning model, optimizing model, quantization model, raspberry pi, real-time detection, apache kafka.

I. INTRODUCTION

The rapid growth of IoT devices has transformed industries such as smart homes, healthcare, industrial automation, and transportation. However, this increased connectivity also exposes IoT networks to a wide range of cyberattacks, including Distributed Denial of Service (DDoS), malware, and unauthorized access [1]. The security of these networks is of paramount importance, as breaches can lead to significant financial losses, privacy violations, and even physical harm in critical systems. To mitigate the risks of such attacks, it is essential to implement effective Intrusion Detection Systems (IDS) to safeguard IoT environments. IDS plays a critical role in continuously monitoring network traffic, identifying malicious activities, and triggering alerts or countermeasures to protect against potential threats [2].

Traditional IDS methods, such as rule-based or signature-based approaches, often struggle to detect evolving and sophisticated cyber threats [3]. In contrast, machine learning (ML) and DL-based IDS models offer a powerful solution,

The associate editor coordinating the review of this manuscript and approving it for publication was Ye Liu.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

capable of identifying new attacks by learning patterns from network data. Among these, ensemble models that combine multiple algorithms have shown excellent performance. Specifically, LSTM networks, a type of recurrent neural network (RNN) excel in analyzing sequential data like network traffic due to their ability to capture temporal dependencies [3], [4]. Stacked LSTM architectures enhance this capability by learning hierarchical data representations. Feature selection techniques like Analysis of Variance (ANOVA) further improve model efficiency by identifying the most relevant features for attack classification.

Our previous work successfully developed an ensemble-based IDS model, which combines stacked LSTM networks with ANOVA feature selection to enhance detection performance while reducing feature complexity. The model was trained on the UNSW-NB15 dataset and achieved impressive results, including an accuracy of 97.46%, had a model size of 2,581 KB, and an overfitting value of just 0.22% [3]. These results demonstrate the effectiveness of our design in identifying malicious traffic with high precision and generalization.

However, while the original model performed well in experimental settings, it also exposed a critical limitation: the model is too computationally intensive for deployment on low-power, memory-limited IoT platforms such as the Raspberry Pi 3B+. It was not optimized for deployment on the Raspberry Pi 3B+. These devices, commonly used in IoT environments, have limited processing power, memory, and energy resources, making it impractical to run sophisticated DL models without significant optimization. Their CPUs are much slower than those used for training, and they often lack sufficient memory to store large models, creating obstacles for efficient real-time processing.

The focus of this current research is to extend our previous work by addressing the challenges of deploying the model on the Raspberry Pi 3B+, optimization is necessary to ensure smooth implementation on the device [5]. Our current focus centers on the most critical factors influencing real-world deployment: CPU utilization, model accuracy, and inference time. These three components represent the primary bottlenecks when executing deep learning models on resource-constrained platforms such as IoT and edge devices. Addressing these challenges is essential to ensure efficient, reliable, and scalable intrusion detection in practical environments. The Raspberry Pi 3B+, with only 1 GB of RAM and a modest ARM processor, cannot accommodate complex models without careful optimization. Therefore, our primary objective at this stage is to ensure that the optimized model can run smoothly within the hardware constraints of the Raspberry Pi. To achieve this, we apply a series of optimization techniques, such as pruning, quantization, and model conversion using TensorFlow Lite, to reduce both the memory footprint and the computational load while preserving detection performance [6]. TensorFlow Lite provides tools specifically designed for optimizing and deploying machine learning models on edge devices [7].

In addition, a critical aspect of developing effective intrusion detection systems is the ability to evaluate them under realistic attack conditions. Apache Kafka, a distributed streaming platform, offers an ideal solution to simulate realistic attack scenarios for the evaluation of IDS [8]. By using Kafka's producer-consumer model, we can create realistic test environments that simulate multiple attackers with different attack patterns, adjust traffic volumes to test system performance under varying loads, and control the timing and intensity of attacks to assess detection speed.

This paper presents an optimized ensemble deep learning approach for real-time intrusion detection specifically designed for resource-constrained Raspberry Pi devices. Our key contributions include:

1) We optimize an ensemble-based IDS model by integrating stacked LSTM with ANOVA feature selection for deployment on the Raspberry Pi 3B+. After converting the model to TensorFlow Lite (TFLite) and applying quantization and pruning techniques, we reduce the model size from 2581 KB to 235 KB, achieving a 90.9% reduction while maintaining 97.26% accuracy. Additionally, we deploy the optimized model to the Raspberry Pi 3B+, achieving loading times of only 223.8ms and inference times of just 2.4ms.
2) We adopt Apache Kafka to emulate network traffic by reading a dataset and streaming it in real-time, enabling the analysis of IDS performance under realistic conditions. We test various scenarios, including different numbers of attackers (one, two, and three), to evaluate how the system handles varying levels of malicious activity.
3) We investigate the impact of the number of Kafka producers, representing the number of attackers, and simulation delays, which determine how quickly attacks affect IDS efficiency. The system's efficiency is evaluated based on its ability to classify traffic quickly and accurately, while also considering hardware performance metrics such as CPU usage.

The rest of this paper is organized as follows: Section II reviews related work, discussing existing methods for intrusion detection. Section III presents our proposed method, covering the design of optimization techniques and real-time implementation. Section IV outlines the experiments and results, including the experiment design, scenarios, performance evaluation, and findings. Finally, Section V concludes the paper, summarizing the key results and offering suggestions for future research.

II. RELATED WORK

In recent years, the integration of ML and DL into IDS has gained significant research attention. To overcome the limitations of conventional IDS approaches, new solutions leverage ensemble learning techniques to enhance detection accuracy and performance. Studies have demonstrated that ensemble methods yield more precise and reliable results

compared to single-model approaches, making them a promising direction for improving IDS effectiveness.

However, deploying IDS on resource-constrained edge devices, such as Raspberry Pi, introduces additional challenges, including limited computational power, memory constraints, and energy efficiency concerns. These factors can hinder the real-time execution of complex ML and DL models. Addressing these challenges requires the development of lightweight models and optimization techniques tailored for resource-limited environments. A comprehensive survey of existing ML and DL-based IDS solutions is provided by [9]. This section explores key studies that focus on enhancing IDS capabilities, particularly in real-time detection and edge computing scenarios.

In [10], traditional machine learning techniques such as random forest (RF) and extreme gradient boost (XGB) are often preferred for their robustness and efficiency in handling large-scale IoT datasets. To address the challenge of class imbalance, the Synthetic Minority Oversampling Technique (SMOTE) has been employed to enhance model performance, particularly in identifying rare IoT attack patterns. Apache Spark and PySpark have been used to manage the processing of large volumes of IoT traffic data, enabling real-time data processing at scale. The experiment utilized the IoT-23 dataset, which consists of network traffic collected from smart home IoT devices, to develop and evaluate the proposed model.

The XGB achieved a high accuracy of 98.89%, demonstrating strong performance in classifying IoT network traffic data. In terms of efficiency, RF exhibited the fastest training and prediction times, with a prediction time of just 0.0311 seconds. However, this study did not evaluate the performance of IDS against large-scale and high-speed traffic attacks, which are critical challenges. The requirement for fast inference times, efficient use of computing resources, and the ability to manage large-scale attacks can introduce scalability issues and potential delays.

In another study [5], ML and DL become an important role in improving IDS for IoT security. CNN with multiple layers was used for deploying a model. Given the hardware limitations of these devices, several optimizations were implemented to ensure efficiency. This study utilized TFLite Model Optimization (TMO) and evaluated it using key criteria, such as model size, accuracy, and inference time to determine the most appropriate DL model for several edge devices, including Raspberry Pi 4, Raspberry Pi 3B+, Arduino Portenta H7, ESP-WROOM-32, NodeMCU V3 and Arduino UNO.

The optimized model performed best on the Raspberry Pi, achieving high accuracy (up to 99.74%) with minimal inference time (as low as $1\mu s$ for attack prediction). However, the study's limitation lies in the fact that it doesn't account for large-scale traffic scenarios, scalability challenges, or potential delays. This leaves open questions about how the model would perform when faced with more diverse, high-volume traffic, which is a critical consideration for real-world IoT deployments.

ML and DL techniques are used in Intrusion Detection Systems (IDS) to handle large, fast, and diverse network traffic. Wong et. al compares ML models like Random Forest (RF), Support Vector Machine (SVM), and k-Nearest Neighbors (kNN) with a hybrid CNN-LSTM model for IDS. Tested on NSL-KDD, ISCX-IDS, and CTU-13 datasets, the CNN-LSTM model outperformed traditional ML methods, achieving 97.2% accuracy with low latency (68ms) [11], proving its efficiency for real-time detection. Optimization techniques such as transfer learning and model compression were explored but not fully evaluated. Additionally, the model's performance in real-time, large-scale traffic remains uncertain due to limited testing.

The integration of IoT in smart cities and critical infrastructure has increased the demand for secure, real-time monitoring systems. Traditional cloud-based IDS often fail to meet the latency, confidentiality, and scalability requirements of modern IoT networks. To address these limitations, researchers have explored edge computing (EC) and lightweight deep learning models. One such study proposed the IoT-Defender framework, which integrates a Modified Genetic Algorithm (MGA) for feature selection and hyperparameter tuning with a deep LSTM network for detecting IoT-based cyberattacks. The use of a focal loss function further addresses class imbalance issues. Evaluated on BoT-IoT, UNSW-NB15, and N-BaIoT datasets using a Raspberry Pi device, the model demonstrated high accuracy (99.41%), detection rate (99.78%), and efficient performance, confirming its suitability for real-time deployment on edge devices [12].

A hybrid deep learning model (CNN-LSTM) was implemented to develop an anomaly-based Intrusion Detection System (IDS) for fog computing environments. The proposed energy-efficient IDS was evaluated on CICIoT2023, KDD-99, and NSL-KDD datasets, achieving over 99% accuracy and a false alarm rate below 0.38% across all datasets. To assess real-world feasibility, the model was deployed on a Raspberry Pi fog node, demonstrating high accuracy and low latency with a maximum power consumption of 6.12 W. However, its performance depends on hardware capabilities, and large datasets increase computational complexity, requiring a balance between speed and accuracy for optimal deployment [13].

A One-Class Bidirectional GRU Autoencoder and Ensemble Learning is utilized to enhance intrusion detection in IoT environments. The autoencoder is trained on normal data to detect anomalies based on reconstruction loss, while ensemble learning with Soft Voting classifies unknown attacks as the most similar known type. Tested on WSN-DS, UNSW-NB15, and KDD99 datasets, the model achieved high recognition rates of 97.91%, 98.92%, and 98.23%, significantly outperforming traditional IDS methods. The approach is efficient, portable, and effective against zero-day

TABLE 1. Comparative study.

Authors, Year	*Methodology*	*Optimization technique*	*Realtime detection*	*Dataset*	*Evaluated parameter*
A. Alrefaei and M. Ilyas, 2024	LR, RF, DT, KNN, XGB	✗	✓	IoT-23 dataset	Evaluated accuracy (XGB: 98.89%) and prediction time (RF: 0.0311s). Lacks large-scale attack testing, has scalability and resource issues, and does not include attack simulation, device implementation, or CPU evaluation.
Idrissi, I. et al., 2022	DL-HIDS with CNN on edge devices	TMO	✗	MQTT IOT-IDS2020 dataset	Evaluated accuracy (up to 99.74%), inference time, and hardware comparison. Limited scalability and generalization; lacks large-scale traffic tests, attack simulation, and CPU usage analysis.
Wong, M. L., and Arjunan, T., 2024	RF, SVM, kNN, CNN, LSTM	✗	✓	NSL-KDD, ISCX-IDS, CTU-13	Evaluated accuracy (97.2%) and latency (68ms). Faces high computational cost, limited scalability, and lacks CPU usage evaluation.
Saheed, Y. K. et al., 2024	Combined MGA-LSTM	✗	✗	BoT-IoT, UNSW-NB15, and N-BaIoT	Evaluated accuracy (99.41%), detection rate (99.78%), precision (98.50%), false alarm rate (2.56%), mean intersection over union (mIoU) of 0.68, and training time of 81.3 seconds. Faces lack real-time validation and provide no CPU or device implementation details.
Hawazen, A. et al., 2024	Hybrid CNN-LSTM DL model	✗	✓	CICIoT2023, KDD-99, NSL-KDD	Evaluated accuracy (>99%), false alarm rate (<0.38%), and power usage (6.12W). Needs further optimization for large datasets and lacks CPU usage analysis.
Yao, Wenbin et al., 2023	One-Class Bi-GRU Autoencoder + Ensemble	✗	✗	WSN-DS, UNSW-NB15, and KDD99	Evaluated accuracy (97.91–98.92%). Struggles with evolving threats, relies on normal data quality, lacks real-time validation, and provides no CPU or device implementation details.
Vinayakumar, R. et al., 2019	Deep Neural Networks (DNNs)	✗	✓	KDDCup99, NSL-KDD, UNSW-NB15, Kyoto, WSN-DS, CICIDS-2017	Evaluated accuracy (95–99% on NSL-KDD, 65–75% on UNSW-NB15). Limited performance on modern datasets, high computational cost, no real-world testing, and lacks device/CPU evaluation.
Chandan, M. et al., 2024	RF, DT, GB, KNN, BNB	✗	✓	NSL-KDD	Evaluated accuracy (RF: 99.59%, BNB: 89.48%). High resource requirements, poor performance with novel attacks, lacks real-time traffic evaluation, device implementation, and CPU usage details.
Karunamurthy, A. et al., 2025	FL trained by using CNN models	✗	✗	MQTT	Evaluated a recall of 93.30%, precision of 94.79%, and an F1-score of 94.04%, showing it can detect attacks accurately with few false alarms. The overall accuracy was 95.59%, lacks real-time traffic evaluation, device implementation, and CPU usage details.
Musthafa, M. B. et al. [Proposed]/2025	Stacked LSTM and optimized it using quantization and pruning	✓	✓	UNSW-NB15	The Stacked LSTM model had 97.46% accuracy, which dropped slightly by 0.21% after optimization. The optimization also reduced model size, loading time, and inference time. The model was deployed on a Raspberry Pi 3B+ and tested under various attack scenarios, showing good real-time performance with peak CPU usage of 111.8% using multi-core processing.

attacks, making it a strong candidate for real-world deployment [14].

IDS was developed by Vinayakumar et al. to detect and classify cyberattacks at both the network and host levels using deep neural networks (DNNs). However, significant challenges are presented by evolving attack methods and the large volume of malicious traffic, requiring scalable solutions. The effectiveness of DNNs in IDS is explored by evaluating benchmark datasets, including KDDCup 99, NSL-KDD, UNSW-NB15, Kyoto, WSN-DS, and CICIDS-2017. A hyperparameter-tuned DNN model was trained for up to 1,000 epochs with learning rates ranging from 0.01 to 0.5, and its generalizability was tested across multiple datasets.

Through an experimental evaluation, it was confirmed that DNNs outperform classical machine learning classifiers in detecting cyberattacks. The results indicate that for the KDDCup 99 and NSL-KDD datasets, most DNN network topologies achieved training accuracy between 95% and 99%. In contrast, for the UNSW-NB15 and WSN-DS datasets, all DNN network topologies exhibited lower training accuracy, ranging from 65% to 75%. Finally, a scalable hybrid framework, Scale-Hybrid-IDS-AlertNet, is proposed for real-time network and host-level monitoring, where distributed deep learning is leveraged to enhance cybersecurity defenses [15].

IDS was utilized to detect anomaly traffic, designed for real-time anomaly detection using Apache Kafka and machine learning. Kafka was used for data streaming. Adopting a holistic approach that combines machine learning algorithms, real-time data analysis, and proactive alerting systems. Network anomalies were detected and classified using a Random Forest Classifier, which is trained on the NSL-KDD dataset. By integrating machine learning algorithms with real-time data processing and automated threat notifications, the system enhances network security and operational continuity. The Random Forest model achieved a high detection accuracy of 99.59% [16].

An optimal Federated Learning (FL)-based IDS was proposed for IoT environments, aiming to address key

challenges such as scalability, data privacy, and limited computing resources. The system architecture includes multiple distributed IoT devices connected through security gateways. Each gateway uses a local CNN-based intrusion detection model to analyze network traffic directly at the source. This decentralized setup keeps raw data on the device, protecting privacy, and enabling fast threat detection. The FL framework then combines updates from each local model into a global model, improving overall detection accuracy across the network. To enhance efficiency, the system uses the Chimp Optimization Algorithm (ChOA) for feature selection. Experiments using the MQTT benchmark dataset show that this FL-based IDS achieves a maximum detection accuracy of 95.59%, outperforming traditional machine learning methods [17].

Table 1 provides a comprehensive summary of major research studies in the IDS domain that utilize ML and DL algorithms, highlighting their real-time implementation and performance evaluation. Our approach addresses the limitations of existing methods by optimizing the IDS model to implement it on Raspberry Pi 3B+ smoothly and enhance performance, particularly by minimizing inference time during real-time detection. Additionally, it effectively handles parallel attacks from multiple sources, ensuring a robust, adaptive, and scalable solution for network intrusion detection.

III. OVERVIEW OF ORIGINAL IDS MODEL

In our earlier study, we proposed an optimized IDS framework that incorporates class balancing and feature selection during preprocessing. The approach was evaluated using the UNSW-NB15 dataset, employing two ensemble models: a Support Vector Machine (SVM) with bagging, and a Long Short-Term Memory (LSTM) model with stacking. The stacking-based LSTM model, when combined with Analysis of Variance (ANOVA) for feature selection, outperformed the SVM-based model across several key metrics. Specifically, it achieved an F-measure of 96.02%, an accuracy of 97.46%, a minimal overfitting rate of 0.22%, and an AUC of 0.9728, reflecting near-perfect class separation and robust intrusion detection capabilities.

The ensemble-based IDS integrated stacked LSTM networks with ANOVA-driven feature selection to effectively capture temporal patterns in network traffic while minimizing input dimensionality [3], [4]. The UNSW-NB15 dataset, which includes a broad spectrum of modern attack types, was used to evaluate model performance. ANOVA was applied to assess the statistical relevance of each feature to the target variable, resulting in a refined set of 36 high-impact features. This reduced feature set contributed to lowering noise and improving both efficiency and accuracy.

The final stacked LSTM model achieved a classification accuracy of 97.46%, with an overfitting rate of just 0.22%, indicating strong generalization and robustness. However, despite its promising detection performance, the model's deployment on resource-constrained platforms posed notable challenges. The final model size of 2,581 KB, along with its computational intensity, rendered it less suitable for real-time applications on edge devices such as the Raspberry Pi. These devices are limited in memory, processing power, and energy capacity, highlighting the need for further optimization to enable lightweight, real-time intrusion detection at the network edge.

IV. PROPOSAL OF OPTIMIZED ENSEMBLE IDS MODEL

This section describes our approach to optimizing and deploying an ensemble deep learning model for intrusion detection on resource-constrained Raspberry Pi devices. We first provide an overview of the original model, followed by the optimization techniques used to reduce its size and computational requirements, and the deployment. Finally, we detail the real-time implementation process.

A. OPTIMIZATION TECHNIQUE

The ensemble-based IDS model is optimized to operate efficiently within the limited memory capacity of edge devices, ensuring low latency and fast response times. This optimization, however, involves a trade-off between model size and accuracy. We have carefully balanced this trade-off to maintain effective intrusion detection capabilities while adhering to the constraints of resource-limited devices.

To enable efficient deployment, we utilized TensorFlow Lite (TFLite) as the primary framework for model optimization and execution on edge platforms. TFLite offers a lightweight interpreter specifically designed for low-power and fast inference on devices such as Raspberry Pi, microcontrollers, and smartphones [18].

One of TFLite's key advantages is its built-in support for model compression techniques, including post-training quantization, which significantly reduces model size and computational complexity, thereby making it highly suitable for deployment in constrained environments [19]. While MobileNet refers to a specific convolutional neural network architecture and TinyML represents a broader paradigm focused on deploying machine learning on resource-constrained devices, TFLite offers a practical and comprehensive framework. Table 2 summarizes the key strengths and limitations of each.

Furthermore, TFLite is scalable across a variety of hardware platforms, supporting multiple hardware accelerators including CPUs, GPUs, and Google's Coral Edge TPU. This versatility allows the same model to be efficiently deployed on devices ranging from microcontrollers to edge computing platforms like the Raspberry Pi.

Figure 1 illustrates our optimization pipeline for the ensemble-based IDS model. The process involves converting the original model, applying quantization and pruning techniques, and deploying the optimized version on the Raspberry Pi 3B+ for real-time inference.

1) CONVERTING ORIGINAL MODEL

The first step is converting the Ensemble-based IDS model, trained using TensorFlow and saved as IDSmodel.h5, into

TABLE 2. Comparison of TFLite, MobileNet, and TinyML.

Item	*Pros*	*Cons*
TFLite	• Lightweight inference engine for edge devices like Raspberry Pi [20] • Supports quantization, pruning, and model optimization [19] • Cross-platform with CPU, GPU, and TPU support [21]	• Limited support for dynamic operations compared to full TensorFlow • No training on-device • Requires additional optimization for peak performance
MobileNet	• Highly efficient CNN for mobile/embedded vision applications [22], [23] • Good trade-off between accuracy and inference speed • Easily convertible to TFLite format for deployment	• Primarily suited for vision tasks, not general ML • Still too heavy for some microcontrollers without quantization • Requires a runtime like TFLite for deployment
TinyML	• Enables ML on ultra-low-power devices [18], [24] • Supports real-time, on-device inference • Enhances privacy and reduces latency by avoiding cloud dependency	• Limited memory, power, and computational resources [19], [25] • Inference-only; training must occur off-device • Often sacrifices model complexity and accuracy due to compression

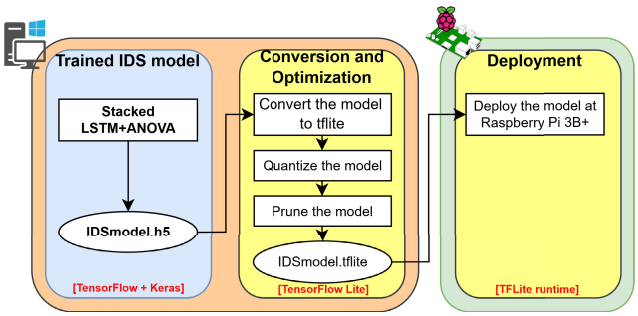

FIGURE 1. Optimization pipeline of the ensemble-based IDS model.

a format suitable for the Raspberry Pi [3], [26]. This format is TFLite, a lightweight version of TensorFlow designed for edge devices [21]. It is a cross-platform DL framework that converts TensorFlow pretrained models into an optimized format in speed and storage, that can be deployed on edge devices like Linux based embedded such as Raspberry Pi.

The conversion is performed using the *TFLiteConverter* function [26]. TFLite is widely preferred for deploying models on devices like the Raspberry Pi due to its compatibility and efficiency. By converting the model to TFLite, we reduce its size and improve its performance, making it ideal for devices with limited resources. This step ensures the model runs smoothly on low-power devices while still providing accurate and fast results.

2) OPTIMIZING THE MODEL

Quantization is one of the most effective methods to compress neural networks and to accelerate the inference of networks. Quantizing a network means reducing the precision of the weights and activations of the network to make it more computationally efficient while maintaining a similar level of prediction accuracy. The quantized weights and activations are typically stored as integers (such as INT8 and INT4), which reduces the amount of memory required to store them. Lower precision calculations can be performed faster than higher precision calculations, resulting in faster inference times and reduced energy in real-time applications. There are two main types of quantization used for neural networks: Post-Training Quantization (PTQ) and Quantization Aware Training (QAT) [27]. In this research, we apply PTQ to the Ensemble-based IDS model.

PTQ is a technique used to optimize machine learning models by converting the weights and/or activations from high precision (typically 32-bit floating point) to lower precision (such as 8-bit integers) [28], [29]. This is achieved by reducing the precision of parameters from floating-point numbers to lower precision integers. This decreases model size and increases inference speed while maintaining accuracy. The goal is to make it more suitable for deployment on resource-constrained devices like Raspberry Pi.

We enabled optimization using $tf.lite.Optimize.DEFAULT$, which applies general latency and size improvements. Furthermore, we specified support for both TFLite built-in operators and selected TensorFlow operators to preserve model functionality. The final quantized model operates with $INT8$ precision, significantly reducing the model size and inference time without substantial loss in accuracy.

Pruning is a process of removing unnecessary or less important components from a neural network, such as weights, neurons, or even entire layers [30], [31]. The primary goals are to decrease the model size and improve computational efficiency without significantly compromising its performance [32]. In this work, we apply *prune_low_magnitude* function. It focuses on reducing the size of a neural network by pruning weights with the smallest magnitudes (insignificant), effectively removing those deemed less significant to the model's performance [33]. This technique involves several parameters that control how pruning is applied to a model. The detail parameter is shown in Table 3.

TABLE 3. Pruning parameters.

Parameters	*Value*
Pruning Schedule	PolynomialDecay
Optimizer	Adam
Initial Sparsity	0.0
Final Sparsity	0.5
Begin Step	2,000
End Step	10,000
Frequency	100

The pruning configuration presented in Table 3 is designed to achieve an effective trade-off between model compression and performance stability. The use of a *PolynomialDecay* schedule allows sparsity to increase gradually from an initial value of 0.0 to a final value of 0.5, enabling the model to adapt smoothly as weights are pruned. This approach helps prevent sudden degradation in accuracy by avoiding abrupt structural changes during early training. Pruning is scheduled to begin at step 2,000 and end at step 10,000, providing a sufficient window for gradual adaptation and recovery. By delaying pruning until after the initial learning phase, the model is allowed to establish strong baseline representations before any reduction in capacity occurs.

Additionally, pruning is applied at a frequency of every 100 steps, which ensures incremental sparsity updates and contributes to training stability. The Adam optimizer is selected for its robustness in handling sparse gradients and adaptive learning rates, both of which are beneficial when weights are being selectively removed. This combination of controlled pruning and a resilient optimizer ensures that the model maintains high accuracy while significantly reducing its size and computational footprint.

3) DEPLOYING THE OPTIMIZED MODEL

The optimized model then deploying on a Raspberry Pi 3 B+ using the TensorFlow Lite runtime.

B. REAL-TIME IMPLEMENTATION

We implement a real-time traffic analysis system using Apache Kafka to evaluate the optimized IDS model under realistic conditions. Utilizing multiple attackers in the simulation is essential to mimic the complexity and concurrency of real-world cyber threats, where different types of attacks often occur simultaneously across a network. Each Kafka producer represents a distinct attacker, generating traffic that includes diverse attack types such as DDoS, SQL injection, and brute force attacks derived from the UNSW-NB15 dataset. This multi-attacker setup provides a comprehensive evaluation of the IDS's ability to detect and classify overlapping threats in real time.

Additionally, introducing Kafka delays between message transmissions simulates realistic network latency and traffic behavior. This controlled timing enables a more accurate assessment of the model's responsiveness and performance in scenarios where data does not arrive in a perfectly sequential or instantaneous manner, as is typical in practical deployment environments. Together, the use of multiple attackers and controlled Kafka delays ensures a robust, real-world emulation for validating the IDS on a Raspberry Pi 3 B+.

Figure 2 illustrates the architecture of a real-time intrusion detection pipeline designed for deployment on edge computing devices, specifically the Raspberry Pi 3B+. This system leverages the Apache Kafka distributed streaming platform to process simulated network traffic in real time. The primary objective is to evaluate the performance of an ensemble-based IDS under conditions that reflect real-world IoT environments.

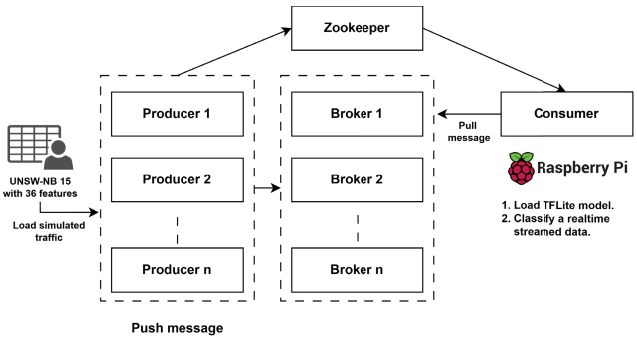

FIGURE 2. Real-time intrusion detection architecture using Apache Kafka and Raspberry Pi 3B+.

Network traffic transactions are generated from the UNSW-NB15 dataset, which contains 36 features per instance and is provided in CSV format. These records are used to simulate realistic network traffic flows. Multiple Kafka producers are employed to load and transmit the traffic into Kafka topics, simulating concurrent sources such as multiple compromised IoT devices. This design enables scalability and provides a mechanism for evaluating the IDS under diverse and overlapping attack scenarios.

Kafka brokers receive the streaming data from producers and manage its distribution using topic partitioning and data replication. This ensures the efficient handling of high-throughput data while maintaining system reliability and preventing data loss. A Zookeeper service is integrated to coordinate the Kafka cluster, manage broker states, and handle leader election and synchronization.

The IDS, embedded in a Raspberry Pi 3B+, functions as a Kafka consumer that continuously pulls data from the brokers. Upon receiving each traffic record, the Raspberry Pi performs real-time inference using an optimized Ensemble-based IDS model to classify each transaction as either normal or attack.

Finally, the classification results are collected and analyzed to assess the system's detection accuracy, inference time, and memory usage. The use of multiple producers and brokers not only enhances fault tolerance and parallel processing but also provides a realistic evaluation environment for intrusion detection in modern, distributed IoT networks.

V. EXPERIMENT AND DISCUSSION

The following section provides a detailed description of the experimental design, including the optimization process, the evaluation scenarios used to assess the performance of the optimized Ensemble-based IDS, and the real-time implementation. Additionally, the section presents the results for each scenario, comparing the performance of the IDS under varying conditions to assess its efficiency, responsiveness, and ability to detect attacks in real time.

A. EXPERIMENT DESIGN

All experiments are conducted using Python as the primary programming language. The experiment consists of two main processes: optimization and real-time implementation, each using a different environment.

The Ensemble-based IDS model is optimized and fine-tuned using TensorFlow Lite, adhering to the same requirements as the model development phase, as shown in Table 6. The optimization process consists of four key steps: model conversion, quantization, pruning, and final integration. First, the trained model is converted into TensorFlow Lite format for efficient deployment. Next, quantization is applied to reduce precision and improve performance, followed by pruning to eliminate redundant parameters while preserving accuracy. Finally, all optimization steps are integrated into a single optimized model for implementation. We evaluate each optimization step to assess its impact on the model's performance. The evaluation includes four key metrics: model size, accuracy, loading time, and inference time.

TABLE 4. Optimization environments.

Hardware/Software	*Specification/version*
CPU	12th Gen Intel(R) Core(TM) i7-12700H processor 2.30 GHz
RAM	64 GB
Disk	1.81 TB
OS	Windows 11 Enterprise 64-bit
Python	3.10.3
TensorFlow	2.10.0

1) Model size represents the amount of memory the model consumes, which reflects how efficiently it has been optimized. A comparison of the original model and the optimized model showed how much the memory usage has been reduced.
2) Loading time measures how quickly the model is set up and ready to use after being initialized. It indicates the speed at which the model becomes functional, enabling the IDS to analyze and categorize network traffic in real-time for efficient threat detection.
3) Inference time evaluates how fast the model generates predictions by comparing the original and optimized versions, directly impacting the system's responsiveness in detecting and mitigating network attacks.
4) Accuracy evaluates how effectively the deep learning model detects attacks, comparing the original model with the optimized model to assess any changes in detection performance.

This evaluation metrics ensures that optimizations enhance efficiency without compromising detection performance.

For real-time implementation, two environments are used. The first environment runs on Oracle VM VirtualBox with Ubuntu 20.04.6 LTS, where Kafka 2.13-3.7.1 is configured to simulate real-time traffic. In this setup, Kafka simulates an attacker by generating and sending traffic to the Raspberry Pi 3B+ as an IDS. In this setup, Kafka simulates an attacker by generating and sending traffic to the Raspberry Pi 3B+ as an IDS. All Kafka components, including the dataset, producer, and broker, are deployed within VirtualBox. The second environment involves running the optimized Ensemble-based IDS on a Raspberry Pi 3B+, with specifications detailed in Table 5.

TABLE 5. Implementation environments.

Hardware/Software	*Specification/version*
Model	Raspberry Pi 3 Model B+
CPU	64-bit SoC @ 1.4GHz
CPU Cores	4 cores (quad-core)
RAM	1 GB
Disk	32 GB
OS	Debian GNU/Linux, Version 11
Python	2.7.18
TFlite runtime	2.14.0

In this experiment, we evaluate the performance of the IDS under different attack scenarios by measuring four key factors: sending time, inference time, accuracy, and CPU performance. Sending time refers to the duration it takes for the attack traffic to be generated and transmitted from the attacker (Kafka producer) to the IDS (Raspberry Pi 3B+), helping us assess the impact of network conditions on real-time detection. CPU performance assesses the load on the Raspberry Pi 3B+ processor, which is crucial for understanding the system's efficiency and resource utilization during real-time operation.

To analyze these factors, we focus on two scenarios: tuning the number of attackers, where we increase the number of producers (attackers) to examine how the IDS handles higher attack volumes and whether detection efficiency is maintained under heavy traffic, and tuning the simulation delay, where variable delays are introduced in traffic generation to simulate real-world conditions and evaluate how latency affects IDS responsiveness. This comprehensive analysis provides valuable insights into the IDS's performance under varying attack intensities, network conditions, and resource constraints. The details of these scenarios are presented in Table 6.

B. EXPERIMENT SETUP

We begin the optimization process by first converting the original model (OM) to TFLite without applying any optimizations, referred to as the Unoptimized TFLite model (UT), which serves as the baseline for comparison. This

TABLE 6. Experiment scenarios.

Experiment	Simulation delay (ms)	Number of producer	Number of data
1	1	1	90,664
2	1	2	181,328
3	1	3	271,992
4	0.1	1	90,664
5	0.1	2	181,328
6	0.1	3	271,992
7	0	1	90,664
8	0	2	181,328
9	0	3	271,992

serves as a baseline for evaluating the impact of optimization techniques. To improve performance, two key optimization techniques are applied: Quantization (QT) and Pruning (PT). Finally, both quantization and pruning are combined to produce the optimized TFLite model (OT), which represents the most efficient version of Ensemble-based IDS. This final model is expected to have a significantly reduced size, improved loading time, and lower inference latency while maintaining high classification accuracy.

In the deployment phase, data streams are transmitted through Apache Kafka before being processed by TensorFlow Lite, and the experiment evaluates their performance under varying conditions, including different simulation delays and numbers of Kafka producers, using the UNSW-NB15 dataset, as outlined in Table 6.

Initially, data streams are transmitted through Apache Kafka before being processed by TensorFlow Lite. The experiment evaluates the performance of both Apache Kafka and TensorFlow Lite under different conditions. Specifically, various simulation delays and different numbers of Kafka producers are tested using the UNSW-NB15 dataset, as outlined in Table 6.

The experiment evaluates the effect of increasing the number of Kafka producers, simulating scenarios with 1, 2, and 3 producers to analyze their impact on data transmission and system performance. A single producer transmits data sequentially, which may introduce bottlenecks and delays in high-traffic conditions as shown in Figure 3. However, multiple producers transmit data in parallel. This parallel processing allows for faster data handling and improved overall performance, making the system more scalable and capable of managing higher workloads effectively.

```
Seq 90652 Producer 0 Sent at 2025-01-20 20:41:50.726
Seq 90653 Producer 0 Sent at 2025-01-20 20:41:50.737
Seq 90654 Producer 0 Sent at 2025-01-20 20:41:50.747
Seq 90655 Producer 0 Sent at 2025-01-20 20:41:50.758
Seq 90656 Producer 0 Sent at 2025-01-20 20:41:50.768
Seq 90657 Producer 0 Sent at 2025-01-20 20:41:50.784
Seq 90658 Producer 0 Sent at 2025-01-20 20:41:50.798
Seq 90659 Producer 0 Sent at 2025-01-20 20:41:50.809
Seq 90660 Producer 0 Sent at 2025-01-20 20:41:50.820
Seq 90661 Producer 0 Sent at 2025-01-20 20:41:50.832
Seq 90662 Producer 0 Sent at 2025-01-20 20:41:50.843
Seq 90663 Producer 0 Sent at 2025-01-20 20:41:50.855
```

FIGURE 3. Streaming data with single Kafka producers sequentially.

As shown in Figure 4, parallel data transmission effectively distributes attack traffic across Kafka brokers, ensuring a seamless and scalable stream. This reduces congestion and enables the IDS to process real-time network traffic with minimal delay. By utilizing multiple producers, the system leverages Kafka's distributed architecture, optimizing load balancing and minimizing latency.

```
Seq 2764 Producer 1 Sent at 2024-12-21 20:38:12.844
Seq 2796 Producer 0 Sent at 2024-12-21 20:38:12.852
Seq 2788 Producer 2 Sent at 2024-12-21 20:38:12.855
Seq 2765 Producer 1 Sent at 2024-12-21 20:38:12.856
Seq 2797 Producer 0 Sent at 2024-12-21 20:38:12.865
Seq 2789 Producer 2 Sent at 2024-12-21 20:38:12.868
Seq 2766 Producer 1 Sent at 2024-12-21 20:38:12.870
Seq 2798 Producer 0 Sent at 2024-12-21 20:38:12.876
Seq 2790 Producer 2 Sent at 2024-12-21 20:38:12.878
Seq 2767 Producer 1 Sent at 2024-12-21 20:38:12.883
Seq 2799 Producer 0 Sent at 2024-12-21 20:38:12.888
Seq 2791 Producer 2 Sent at 2024-12-21 20:38:12.889
Seq 2768 Producer 1 Sent at 2024-12-21 20:38:12.899
Seq 2800 Producer 0 Sent at 2024-12-21 20:38:12.899
Seq 2792 Producer 2 Sent at 2024-12-21 20:38:12.900
Seq 2801 Producer 0 Sent at 2024-12-21 20:38:12.910
Seq 2769 Producer 1 Sent at 2024-12-21 20:38:12.910
Seq 2793 Producer 2 Sent at 2024-12-21 20:38:12.913
Seq 2802 Producer 0 Sent at 2024-12-21 20:38:12.921
Seq 2770 Producer 1 Sent at 2024-12-21 20:38:12.923
Seq 2794 Producer 2 Sent at 2024-12-21 20:38:12.924
```

FIGURE 4. Streaming data with 3 Kafka producers paralelly.

Additionally, parallel processing allows the IDS to analyze incoming traffic more efficiently, ensuring faster classification of normal and malicious traffic. This scalability is crucial for handling high-volume network environments where real-time detection is essential. IDS can process larger datasets quickly by using multiple Kafka producers, ensuring minimal delays in threat detection even under heavy-attack traffic conditions.

C. EXPERIMENTAL RESULTS

1) MODEL OPTIMIZATION RESULTS AND ITS PERFORMANCE ON RASPBERRY PI

Table 7 summarizes the impact of our optimization techniques on model size, accuracy, loading time, and inference time.

The OT model is the most efficient version of Ensemble-based IDS, performing better than all other models in every evaluation metric. By applying quantization and pruning techniques, it achieves the smallest size, fastest in loading time, and inference time. Accuracy is the most important evaluation metrics with higher values ensuring better threat detection. OT provides the best balance between optimization and accuracy. Additionally, OT had a significant impact on model size, reducing it by 90.9% compared to the original model.

Throughout the optimization process, all performance metrics improved significantly. Inference time was greatly reduced, while accuracy remained nearly unchanged. This shows the optimization successfully enhanced efficiency

TABLE 7. Model optimization result and performance on Raspberry Pi.

Model	Output size (KB)	Accuracy (%)	Loading time (ms)	Inference time (ms)
OM	2,581	97.46	6,655.7	327.5
UT	865	96.93	378.3	2.5
QT	235	96.94	400,9	2.4
PT	865	97.14	241.2	3.2
OT	235	97.26	223.8	2.4

without sacrificing model performance. The UT model optimized with PT and QT achieved the same output size as OT. However, OT delivered the best performance improvements. Notably, the original model took 6,655.7 ms to load the model, while the TFLite model loaded less than 500 ms. OT had the most significant impact, reducing loading time to 223.8 ms with percentage of decreasing model size is 96.94% as shown in Figure 5.

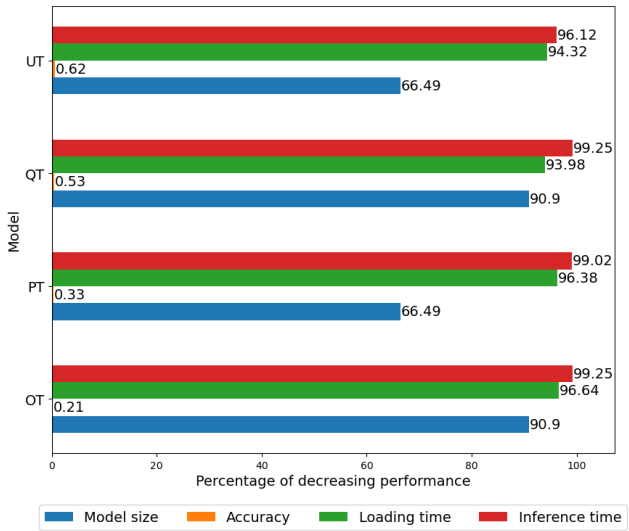

FIGURE 5. Percentage of decreasing performance of each process comparing with original model.

Figure 5 illustrates the comparative percentage reductions in key performance metrics: model size, loading time, and inference time, across four models. These percentages reflect the improvement achieved through various optimization techniques, with the exception of accuracy, where a higher value indicates better model performance and reliability in detecting threats. Among all the models, OT demonstrates the most comprehensive improvements.

OT achieves the highest percentage reductions in model size (90.9%), loading time (96.64%), and inference time (99.25%), highlighting its efficiency and suitability for deployment in latency-sensitive and resource-constrained environments such as IoT-based networks. Despite these reductions, OT maintains a high accuracy of 0.21%, indicating that the optimization process does not compromise the model's ability to detect intrusions effectively. Similarly, quantization TFLite (QT) and PT also show strong performance in reducing inference time while preserving high accuracy (99.25% and 99.02% respectively), though not to the same extent as OT. In contrast, the UT model, which serves as the baseline, lags behind in efficiency metrics while maintaining a slightly lower inference time of 96.12%.

Overall, the results validate that the combination of optimization techniques used in the OT model, including quantization and pruning successfully enhances the practicality of the ensemble-based IDS. This makes OT particularly well-suited for real-time IDS applications in IoT environments where low latency and minimal resource consumption are critical.

2) REALTIME IMPLEMENTATION

The evaluation results for different scenarios, as shown in Table 8, highlight the impact of the number of producers and simulation delay on all evaluation metrics except accuracy. Notably, accuracy remains consistent across all experiments at 97.3%, indicating that variations in attack traffic volume and delay do not affect the IDS's detection capability. However, these factors significantly influence both sending time and inference time, affecting the overall efficiency of the IDS.

An increasing the number of producers leads to a longer sending time and inference time. This is due to the higher volume of attack traffic being generated and transmitted simultaneously, increasing the load on both the Kafka brokers and the IDS deployed on the Raspberry Pi 3B+. As a result, the IDS requires more processing time to analyze the incoming traffic, which can impact real-time threat detection capabilities.

Conversely, reducing the simulation delay improves performance by decreasing both sending time and inference time. When traffic is transmitted with minimal delay, the IDS can process and classify attacks more efficiently. This effect is particularly evident in experiments 7–9 (Figure 6), where no simulation delay is applied. In these cases, both sending and inference times are significantly lower compared to other experiments, demonstrating how minimizing artificial delays improves real-time detection performance.

Increasing the simulation delay causes a gradual rise in both sending time and inference time, as shown in Table 8. This occurs because longer delays slow down the transmission rate of attack traffic to the IDS, leading to an overall reduction in the system's processing speed. A higher simulation delay means that data arrives at a slower rate, which in turn slows down the IDS's ability to analyze and respond to potential threats in real time. This delay can impact the system's efficiency in detecting and mitigating attacks promptly. However, despite variations in attack scenarios, the IDS demonstrated stability and effectively processed network traffic without significant performance degradation.

The system reached a peak CPU utilization of 111.8% across all experiments, indicating that it effectively leverages multi-core processing for real-time intrusion detection. This level of utilization reflects the IDS's ability to distribute

TABLE 8. Experiment results.

Experiment	Simulation delay (ms)	Number of producer	Number of data	Sending time (s)	Average inference time (s)	Total inference time (s)	Accuracy (%)
1	1	1	90,664	180.79	0.001444	130.945467	97.26
2	1	2	181,328	190.29	0.001461	264.988779	97.26
3	1	3	271,992	205.85	0.001539	416.088114	97.26
4	0.1	1	90,664	86.56	0.001427	129.341226	97.26
5	0.1	2	181,328	99.01	0.001450	262.894609	97.26
6	0.1	3	271,992	124.70	0.001541	419.055577	97.26
7	0	1	90,664	21.22	0.001432	129.833945	97.26
8	0	2	181,328	24.89	0.001437	260.634323	97.26
9	0	3	271,992	71.89	0.001550	405.695658	97.26

computational tasks across multiple CPU cores rather than relying on single-core execution. By doing so, the system ensures prompt analysis of incoming network traffic, minimizing bottlenecks and improving response times. Notably, the 111.8% CPU usage was not reached immediately; rather, it increased gradually over time as the workload intensified. In the early stages of execution, CPU usage remained relatively low, but as the number of attack packets grew and more concurrent data streams, such as from two or three Kafka producers were introduced, the system dynamically scaled up its resource usage. This adaptive behavior demonstrates efficient CPU management, where the IDS only escalates resource consumption as needed, preventing early saturation and maintaining headroom for load spikes.

Such scalable and efficient CPU utilization is essential for handling high traffic volumes and complex attack scenarios without significant performance degradation. Even under increased load, the system remained stable and responsive, maintaining real-time processing capabilities. This ensures that threat detection is performed with low latency and that alerts are generated promptly, thereby enhancing the overall reliability, responsiveness, and suitability of the IDS for deployment on constrained edge devices in real-world cybersecurity environments.

The IDS consistently demonstrates high performance and reliability under various testing conditions. Regardless of changes in the number of Kafka producers or variations in simulation delay, the system maintains a stable accuracy of 97.2%, ensuring effective and precise attack detection. While an increase in the number of producers leads to longer sending and inference times due to higher traffic loads, the IDS continues to process network data efficiently without significant performance degradation. Similarly, reducing the simulation delay improves real-time processing by minimizing the time required to detect and respond to attacks. These findings highlight the robustness of the IDS in managing different network traffic scenarios while maintaining its detection capabilities.

Furthermore, the ability of the IDS to function optimally across varying conditions confirms its suitability for real-world deployment, particularly in resource-limited environments. Even when deployed on a low-power device such as the Raspberry Pi 3B+, the system remains responsive and capable of handling real-time attack traffic efficiently. This efficiency is crucial for IoT and edge computing environments where computational resources are constrained. The optimized IDS, with its balance of accuracy, processing speed, and low resource consumption, proves to be a practical solution for securing network infrastructures against cyber threats while ensuring minimal latency and energy consumption.

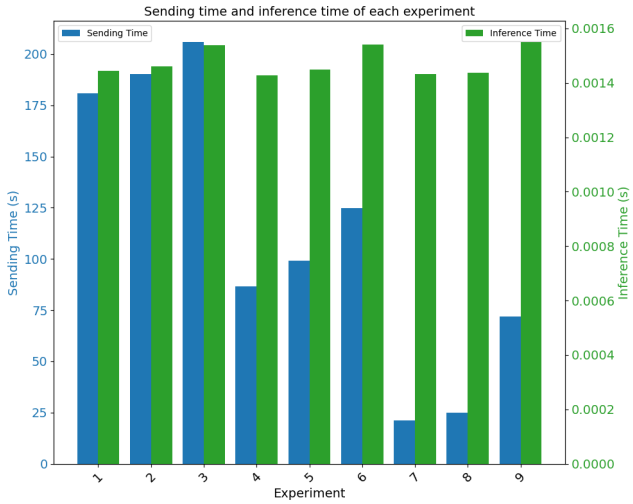

FIGURE 6. Comparison of sending time and inference time each experiment.

VI. CONCLUSION

This study presents an optimized ensemble-based IDS, extending our previously developed stacked LSTM model enhanced with ANOVA-based feature selection, specifically tailored for real-time deployment on low-power edge devices, with a focus on the Raspberry Pi 3B+. The core innovation lies in the integration of quantization and pruning techniques to produce a lightweight performing model. These optimizations resulted in a 90.9% reduction in model size, a 96.94% faster loading time, and significantly reduced inference latency without sacrificing accuracy, which remained high at 97.3%. This demonstrates a practical and scalable balance between computational efficiency and detection accuracy,

setting a new benchmark for deploying IDS on constrained IoT and edge platforms.

A further key contribution of this work is the real-time performance evaluation of the optimized IDS under dynamic attack scenarios using Apache Kafka, where the system maintained consistent performance across varying traffic loads. The model also demonstrated effective multi-core utilization, with CPU usage peaking at 111.8%, underscoring its suitability for real-world edge applications.

One identified limitation is the lack of online model updating. This omission was a deliberate trade-off due to the limited computational and memory resources available on devices like the Raspberry Pi. Online learning introduces complexities such as continual model updates, labeled data requirements, and risks of performance drift, all of which pose challenges in real-time, low-resource settings.

Looking forward, future work will explore lightweight online learning mechanisms and incremental update strategies that can allow the model to dynamically adapt to evolving attack patterns while maintaining low overhead. We also plan to evaluate energy consumption profiles to better understand deployment trade-offs in battery-powered or energy-aware environments. To further enhance scalability, security, and privacy, we aim to integrate blockchain for decentralized trust management, federated learning for distributed privacy-preserving training, and edge-to-cloud orchestration for hierarchical threat detection across heterogeneous infrastructures. Furthermore, while this study was limited to the UNSW-NB15 dataset, future research will expand to other benchmark datasets and apply transfer learning techniques to improve the model's generalization and its ability to detect a wider range of attack types.

REFERENCES

[1] K. Saurabh, S. Sood, P. A. Kumar, U. Singh, R. Vyas, O. P. Vyas, and R. Khondoker, "LBDMIDS: LSTM based deep learning model for intrusion detection systems for IoT networks," in *Proc. IEEE World AI IoT Congr. (AIIoT)*, Jun. 2022, pp. 753–759.

[2] P. Dini, A. Elhanashi, A. Begni, S. Saponara, Q. Zheng, and K. Gasmi, "Overview on intrusion detection systems design exploiting machine learning for networking cybersecurity," *Appl. Sci.*, vol. 13, no. 13, p. 7507, Jun. 2023.

[3] M. B. Musthafa, S. Huda, Y. Kodera, M. A. Ali, S. Araki, J. Mwaura, and Y. Nogami, "Optimizing IoT intrusion detection using balanced class distribution, feature selection, and ensemble machine learning techniques," *Sensors*, vol. 24, no. 13, p. 4293, Jul. 2024.

[4] M. B. Musthafa, S. Huda, M. A. Ali, Y. Kodera, and Y. Nogami, "Evaluation of IDS model by improving accuracy and reducing overfitting using stacking LSTM," in *Proc. IEEE Int. Conf. Consum. Electron. (ICCE)*, Jan. 2024, pp. 1–5.

[5] I. Idrissi, O. Moussaoui, and M. Azizi, "A lightweight optimized deep learning-based host-intrusion detection system deployed on the edge for IoT," *Int. J. Comput. Digit. Syst.*, vol. 11, no. 1, pp. 209–216, Jan. 2022.

[6] J. Kim, S. Chang, and N. Kwak, "Pqk: Model compression via pruning, quantization, and knowledge distillation," in *Proc. Interspeech*, Brno, Czechia, Aug./Sep. 2021, pp. 4568–4572. [Online]. Available: https://www.isca-archive.org/interspeech_2021/kim21m_interspeech.pdf

[7] C. C. Pulgarín-Ospina, L. Launet, A. Colomer, and V. Naranjo, "Optimizing deep learning models for edge computing in histopathology: Bridging the gap to clinical practice," *Proc. Comput. Sci.*, vol. 246, pp. 2549–2557, Jan. 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877050924024840

[8] P. Le Noac'h, A. Costan, and L. Bougé, "A performance evaluation of apache kafka in support of big data streaming applications," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Dec. 2017, pp. 4803–4806.

[9] S. Muneer, U. Farooq, A. Athar, M. Ahsan Raza, T. M. Ghazal, and S. Sakib, "A critical review of artificial intelligence based approaches in intrusion detection: A comprehensive analysis," *J. Eng.*, vol. 2024, no. 1, Jan. 2024, Art. no. 3909173. [Online]. Available: https://onlinelibrary.wiley.com/doi/abs/10.1155/2024/3909173

[10] A. Alrefaei and M. Ilyas, "Using machine learning multiclass classification technique to detect IoT attacks in real time," *Sensors*, vol. 24, no. 14, p. 4516, Jul. 2024.

[11] T. Arjunan, "Real-time detection of network traffic anomalies in big data environments using deep learning models," *Int. J. Res. Appl. Sci. Eng. Technol.*, vol. 12, no. 3, pp. 844–850, Mar. 2024.

[12] Y. K. Saheed, O. H. Abdulganiyu, and T. A. Tchakoucht, "Modified genetic algorithm and fine-tuned long short-term memory network for intrusion detection in the Internet of Things networks with edge capabilities," *Appl. Soft Comput.*, vol. 155, Apr. 2024, Art. no. 111434. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1568494624002084

[13] H. Alzahrani, T. Sheltami, A. Barnawi, M. Imam, and A. Yaser, "A lightweight intrusion detection system using convolutional neural network and long short-term memory in fog computing," *Comput., Mater. Continua*, vol. 80, no. 3, pp. 4703–4728, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1546221824006386

[14] W. Yao, L. Hu, Y. Hou, and X. Li, "A lightweight intelligent network intrusion detection system using one-class autoencoder and ensemble learning for IoT," *Sensors*, vol. 23, no. 8, p. 4141, Apr. 2023. [Online]. Available: https://www.mdpi.com/1424-8220/23/8/4141

[15] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat, and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol. 7, pp. 41525–41550, 2019.

[16] M. S. Chandan, B. K. Soujanya, S. Suman, B. M. Fawaz, and B. S. Roshan, "Kafka-based intrusion detection system," *Grenze Int. J. Eng. Technol. (GIJET)*, vol. 10, Jun. 2024.

[17] A. Karunamurthy, K. Vijayan, P. R. Kshirsagar, and K. T. Tan, "An optimal federated learning-based intrusion detection for IoT environment," *Sci. Rep.*, vol. 15, no. 1, p. 8696, Mar. 2025, doi: 10.1038/s41598-025-93501-8.

[18] P. Warden and D. Situnayake, *TinyML: Machine Learning With TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. Sebastopol, CA, USA: O'Reilly, 2019.

[19] C. R. Banbury, V. Janapa Reddi, M. Lam, W. Fu, A. Fazel, J. Holleman, X. Huang, R. Hurtado, D. Kanter, A. Lokhmotov, D. Patterson, D. Pau, J.-S. Seo, J. Sieracki, U. Thakker, M. Verhelst, and P. Yadav, "Benchmarking TinyML systems: Challenges and direction," 2020, *arXiv:2003.04821*.

[20] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, S. Regev, R. Rhodes, T. Wang, and P. Warden, "TensorFlow lite micro: Embedded machine learning on TinyML systems," 2020, *arXiv:2010.08678*.

[21] (2025). *Tensorflow Lite*. Accessed: Feb. 7, 2025. [Online]. Available: https://www.tensorflow.org/lite/guide

[22] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.

[23] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," 2018, *arXiv:1801.04381*.

[24] A. Anwar, "Recent advances in tinyml hardware and applications," *Electronics*, vol. 12, no. 4, 2023.

[25] A. Elhanashi, "Recent advances in tinyml hardware and application," *Electronics*, vol. 13, no. 17, p. 3562, 2024.

[26] M. B. Musthafa, S. Huda, S. M. Shamim, Y. Kodera, J. Mwaura, S. Araki, and Y. Nogami, "A deployment of ensemble-based IDS on raspberry pi," in *Proc. IEEE 13th Global Conf. Consum. Electron. (GCCE)*, Oct. 2024, pp. 184–185.

[27] H. Wu, P. Judd, X. Zhang, M. Isaev, and P. Micikevicius, "Integer quantization for deep learning inference: Principles and empirical evaluation," 2020, *arXiv:2004.09602*.

[28] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 2704–2713.

[29] B. Rokh, A. Azarpeyvand, and A. Khanteymoori, "A comprehensive survey on model quantization for deep neural networks in image classification," *ACM Trans. Intell. Syst. Technol.*, vol. 14, no. 6, pp. 1–50, Nov. 2023, doi: 10.1145/3623402.
[30] Z. Liu, M. Sun, T. Zhou, G. Huang, and T. Darrell, "Rethinking the value of network pruning," 2018, *arXiv:1810.05270*.
[31] D. Ghimire, K. Lee, and S.-H. Kim, "Loss-aware automatic selection of structured pruning criteria for deep neural network acceleration," *Image Vis. Comput.*, vol. 136, Aug. 2023, Art. no. 104745. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0262885623001191
[32] M. C. Mozer and P. Smolensky, "Using relevance to reduce network size automatically," *Connection Sci.*, vol. 1, no. 1, pp. 3–16, Jan. 1989.
[33] S. Ameen, K. Siriwardana, and T. Theodoridis, "Optimizing deep learning models for raspberry pi," 2023, *arXiv:2304.13039*.

MUHAMMAD BISRI MUSTHAFA received the S.Kom. degree in information systems from the Department of Informatic Engineering, State Islamic University of Malang, Indonesia, in 2013, and the M.Tr.T. degree in data science from the State Polytechnic of Malang, Indonesia, in 2020. He is currently pursuing the Ph.D. degree with the Information Security Laboratory, Okayama University. He has published several research papers in international conferences. His research interests include machine learning, deep learning, intrusion detection systems, and information security.

SAMSUL HUDA (Member, IEEE) received the B.S. degree in telecommunication engineering, the M.E. degree in information and computer engineering from Politeknik Elektronika Negeri Surabaya, Indonesia, in 2014 and 2016, respectively, and the Ph.D. degree from the Graduate School of Natural Science and Technology, Okayama University, Japan, in 2020. He previously worked with Universitas 17 Agustus 1945 Surabaya (known as UNTAG Surabaya), Indonesia, as a Lecturer in Informatics department. Since October 2022, he has been at Okayama University, Japan, as Specially Appointed Assistant Professor with the Green Innovation Center. From April 2025, following the university's structural reorganization, the center is currently part of the interdisciplinary education and research field, where he is currently affiliated. His research interests include computer networks, the Internet of Things (IoT), and network security. He is the Guest Editor of special issues in *Sensors* and *CMC-Computers, Materials and Continua*, and has organized special sessions at ICCE-TW (2024 and 2025) and ICCE 2025, and served as the session chair for several international conferences. He is amember of IAENG.

TUY TAN NGUYEN (Senior Member, IEEE) received the M.S. and Ph.D. degrees in information and communication engineering from Inha University, South Korea, in 2016 and 2019, respectively. He was a Senior Research Engineer with Context Inc., South Korea, from August 2019 to April 2021. From May 2021 to July 2022, he was a Lecturer with the School of Global Convergence Studies and a Postdoctoral Fellow with the Department of Electrical and Computer Engineering, Inha University. He is currently an Assistant Professor with the School of Informatics, Computing, and Cyber Systems, Northern Arizona University, USA. His research interests include post-quantum cryptography, homomorphic encryption, error correction codes, and applied artificial intelligence. He serves as a Technical Committee Member for the IEEE Circuits and Systems Society–Circuits and Systems for Communications.

YUTA KODERA (Member, IEEE) received the degree and the M.E. and Ph.D. degrees from Okayama University, in 2017, 2018, and 2020, respectively. He joined Okayama University, in 2020. His current research interests include applications such as random number generator and cryptography designed over algebraic theory. He is a member of IEICE.

YASUYUKI NOGAMI (Member, IEEE) received the Ph.D. degree from Shinshu University, in 1999. In the same year, he joined Okayama University, as an Assistant Professor, then an Associate Professor, in 2010, and a Professor, in 2017. He is currently the Vice Executive Director for Promotion of Digital Transformation of Okayama University. He served as a member for the IEICE Technical Committee of Information Theory, Information Security, and Hardware Security. He is a member of IEICE. He also served as an Editorial Board Member for *IEICE Transactions Fundamentals* (EA) and a Treasurer and a General Affairs Secretary of IEICE ESS. He is also the Vice-Chairperson of IEICE Technical Committee of the Information Theory.